ASSIGNMENT 4 – TASK 1

Kannada-MNIST Classification Using Recurrent Neural Networks, LSTM and GRU

Student Name: Sushmita Halder, Jyoti, Rajeev Kumar

Roll Number: 2621MA08, 2621MA12, 2621MA01

Group: 08 **Course:** Large Language Models

Submission Date: 12/10/2026

1. Introduction

This project classifies handwritten Kannada digits using recurrent neural networks. Each 28 × 28 grayscale image is treated as a sequence of 28 rows, with 28 pixel features per timestep. Vanilla RNN (SimpleRNN), LSTM, and GRU are compared using training histories, validation performance, test results, confusion-matrix analysis, and input-gradient diagnostics.

2. Objectives

- Prepare sequential Kannada-MNIST image inputs.
- Compare Vanilla RNN, LSTM, and GRU.
- Report training-error curves and explored hyperparameters.
- Evaluate every model (and the validation-selected model) using classification reports and confusion matrices.
- Visualize the original and sequential representation of every class, and at least five samples for each of ten classes.
- Discuss performance, convergence, and gradient flow.

3. Dataset and Preprocessing

The CSV contains 60,000 examples and 785 columns (label plus 784 pixels). Each of the ten classes has 6,000 examples. The input sequence shape is (28, 28). The split is 42,000 training, 9,000 validation, and 9,000 test examples (stratified, random_state = 42).

Property	Value
Dataset size	60,000 images
Image size	28 × 28
Classes	10 (0–9)
Input sequence	28 timesteps × 28 features
Train / validation / test	70% / 15% / 15% (42,000 / 9,000 / 9,000)
Samples per class	6,000

4. Sequential Representation

Each image row is a timestep and each row's 28 pixel values are the input features. A flat 784-pixel vector is reshaped to a (28, 28) array $X = [x_1, x_2, ..., x_{28}]^T$ with x_t in R^{28}, which lets the recurrent layer process the image row by row.

4.1 Reshaping output

Notebook output of the reshape step (Cells 2–3) and the extraction of each row as a time-step input:

```
Original X shape: (60000, 784)
Label shape: (60000,)
Unique labels: [0 1 2 3 4 5 6 7 8 9]

Sequential data shape: (60000, 28, 28)
One image shape: (28, 28)
Time step 1: shape = (28,)
...
Time step 27: shape = (28,)
Time step 28: shape = (28,)
Time step 1 ... Time step 28: each x_t has shape (28,)
```

The split step (Cell 4) then produces the arrays X_train (42000, 28, 28), X_val (9000, 28, 28) and X_test (9000, 28, 28).

4.2 Normalized sequence data and first-row values

After scaling, the sequence array keeps the shape (60000, 28, 28); one image has shape (28, 28) and one time step has shape (28,). The minimum and maximum pixel values are 0.0 and 1.0. The first row of the first image (label 0) is entirely zeros (background), as shown below.

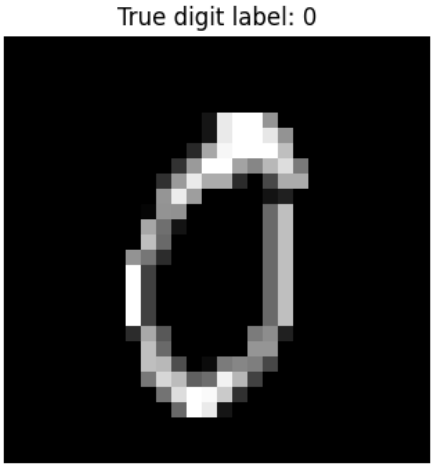


Figure 1. First image (true label 0) after normalization and reshaping.

4.3 Original image versus sequential representation (one sample)

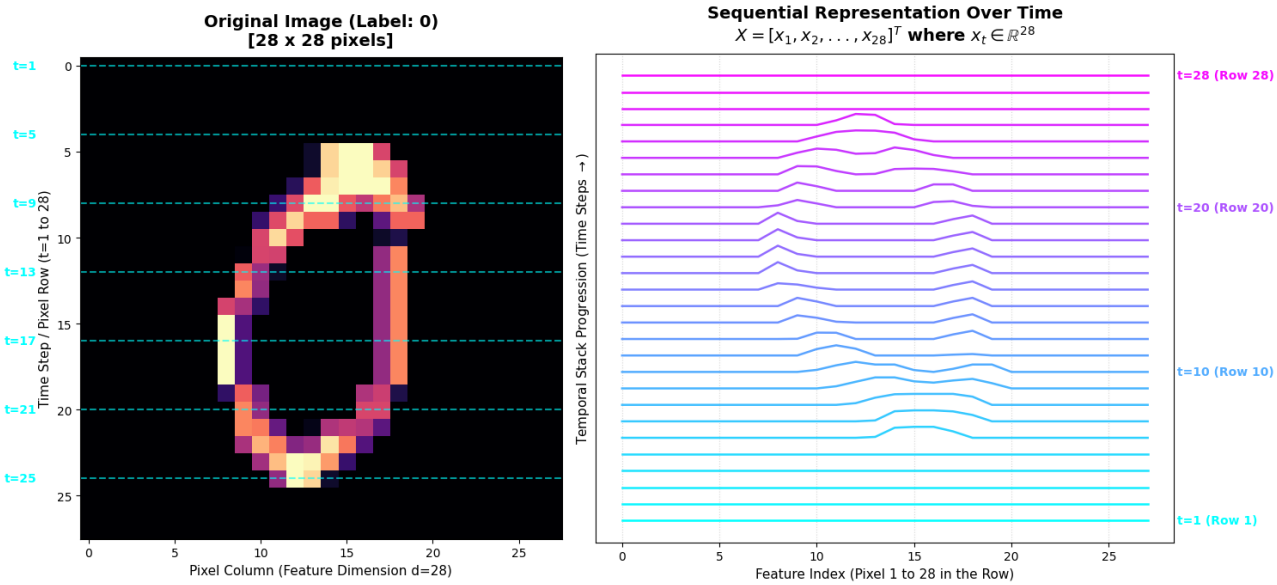


Figure 2. Left: original 28 × 28 image with row partitions (time steps). Right: the same image as a sequence of 28 row vectors stacked over time.

4.4 Original images and sequential representations for all ten classes

Figure 3 shows the original 28 × 28 image for the first example of each label (0–9). Figure 4 shows the corresponding sequential representation of the same ten images: every curve is one timestep (row), plotted over the 28 pixel-column features and shifted upward by its timestep index (dark = early rows, bright = late rows).

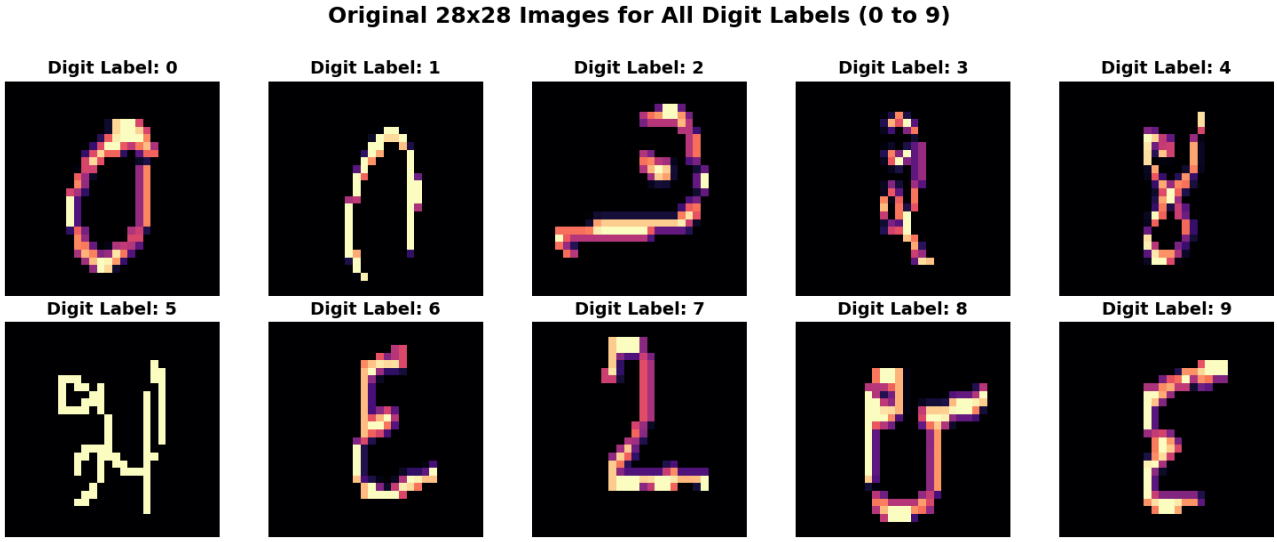


Figure 3. Original 28 × 28 images for all digit labels 0 to 9.

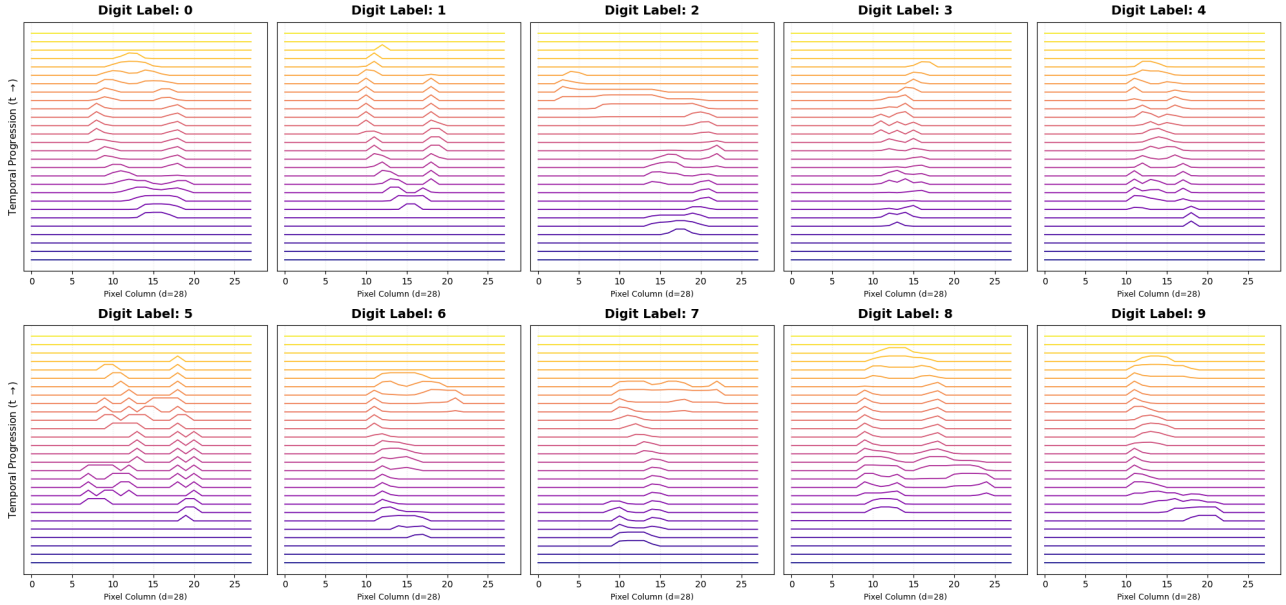


Figure 4. Sequential temporal representation X (28 × 28) for all digit labels 0 to 9 (same ten images as Figure 3).

5. Qualitative Feature Visualizations

Figure 5 includes five 28 × 28 image heatmaps for each digit class, totaling 50 samples.

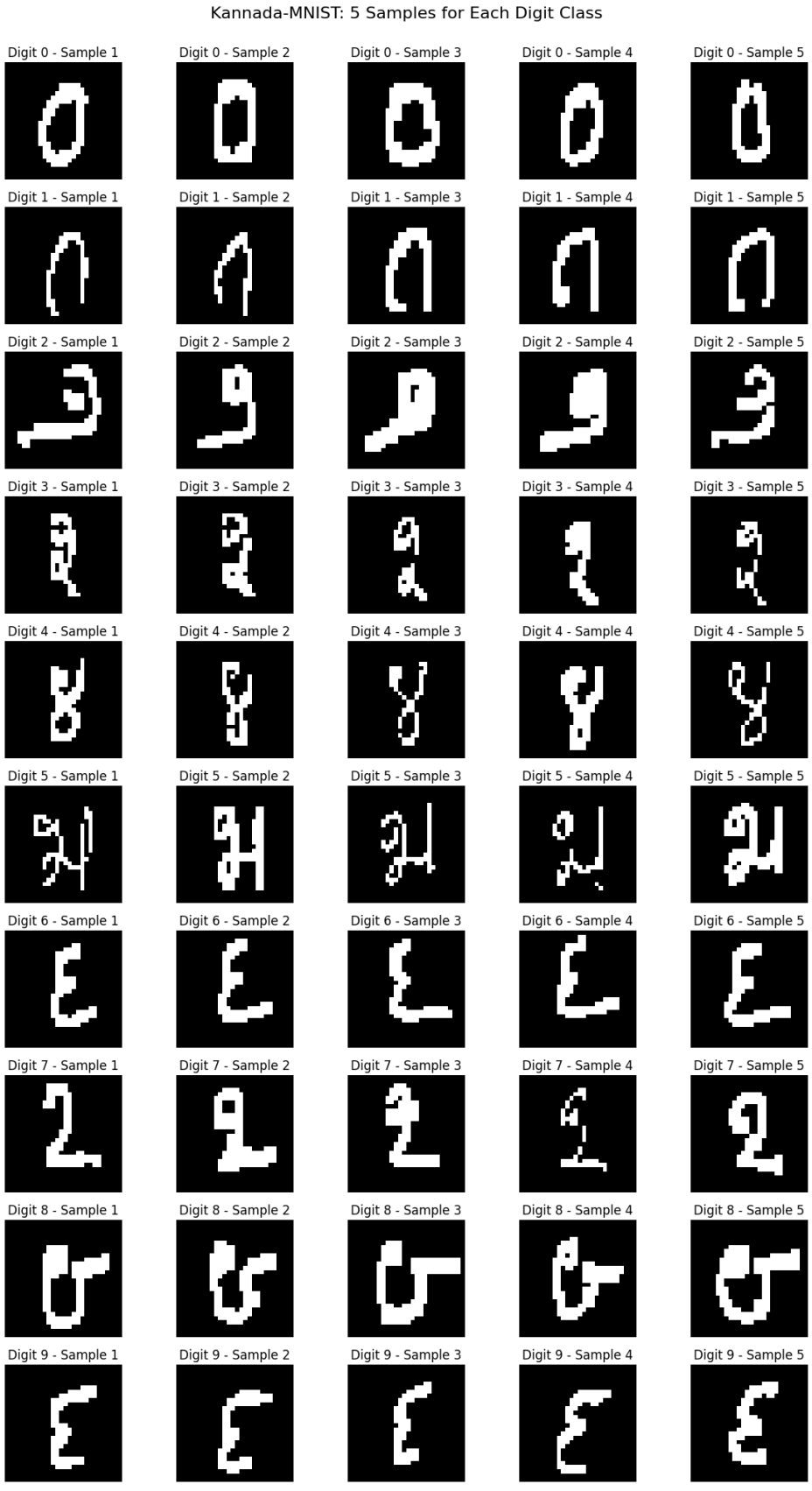


Figure 5. Five sample heatmaps for each class 0–9.

6. Model Setup

Component	Main comparison configuration
Recurrent layer	SimpleRNN, LSTM, or GRU (128 units in the initial run; 64 units in the 8-epoch run)
Regularization / dense	Initial run: Dropout 0.3, Dense(64, ReLU), Dropout 0.2
Output	Dense(10, softmax)
Optimizer / loss	Adam (0.001) / sparse categorical cross-entropy
Batch size	128

7. Training Error and Convergence (a)

7.1 Initial run: 40-epoch budget with early stopping

The first training run (Cell 13) used 128-unit recurrent layers with dropout and callbacks. Vanilla RNN used all 40 epochs and was still improving; LSTM stopped at epoch 17 (best epoch 12) and GRU at epoch 19 (best epoch 14).

Model	Epochs trained	Best epoch (restored)	Validation accuracy
Vanilla RNN	40	40	74.08%
LSTM	17	12	98.01%
GRU	19	14	97.57%

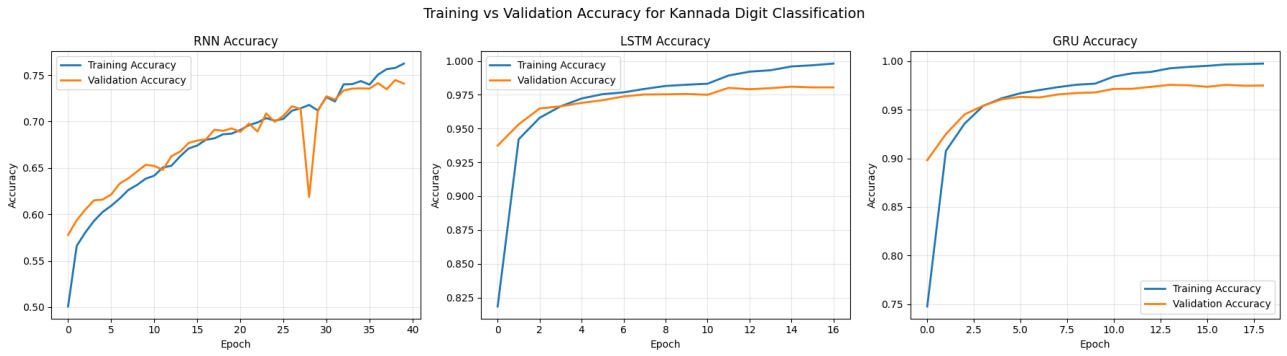


Figure 6. Training vs. validation accuracy per model (initial run, early stopping).

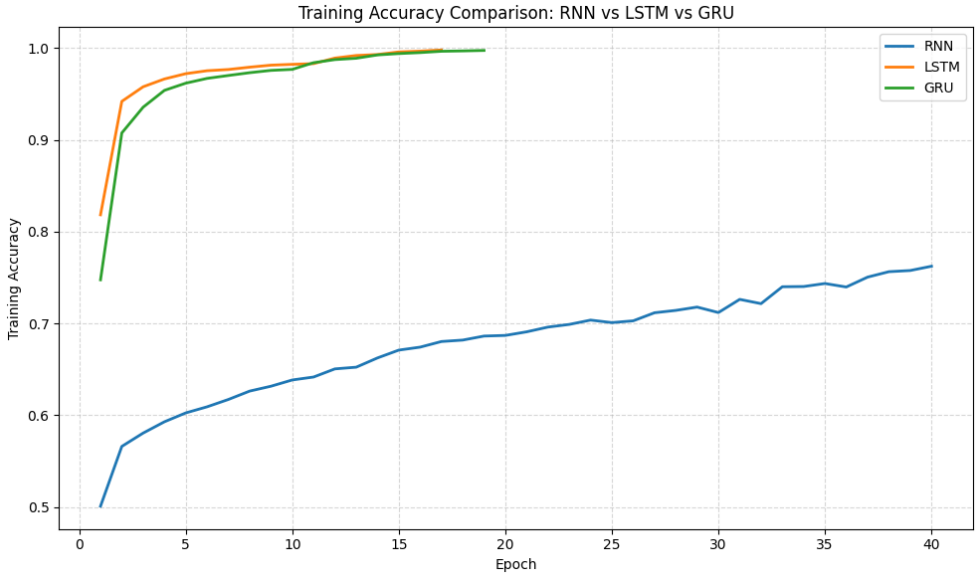


Figure 7. Training accuracy comparison: RNN vs LSTM vs GRU (initial run).

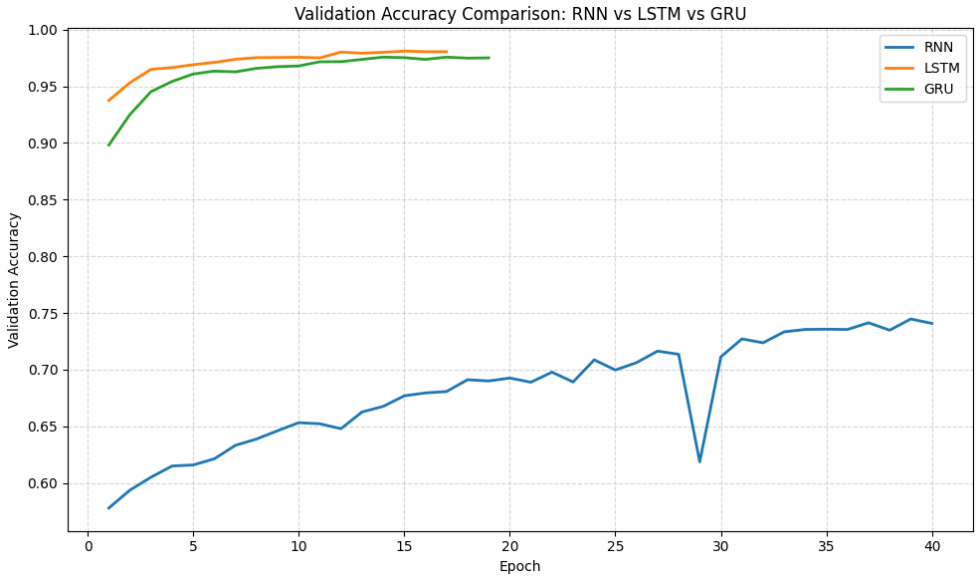


Figure 8. Validation accuracy comparison: RNN vs LSTM vs GRU (initial run).

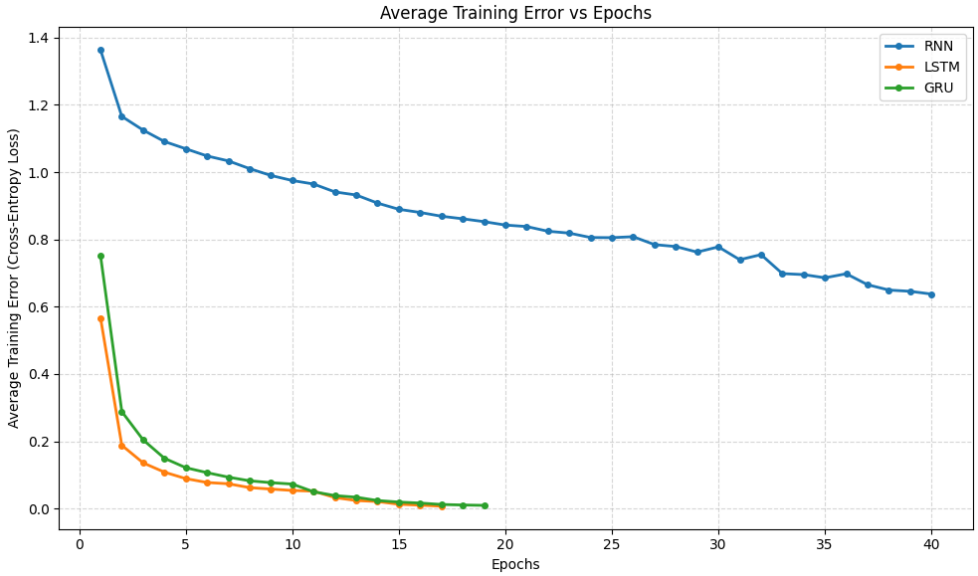


Figure 9. Average training error (cross-entropy loss) vs. epochs (initial run).

7.2 Controlled 8-epoch comparison (64 units, no dropout)

The explicit eight-epoch comparison (Cell 19) trains all three models under identical settings. The short hyperparameter sweep did not store epoch histories for each of its six configurations, so per-configuration loss curves cannot be reconstructed from the saved outputs.

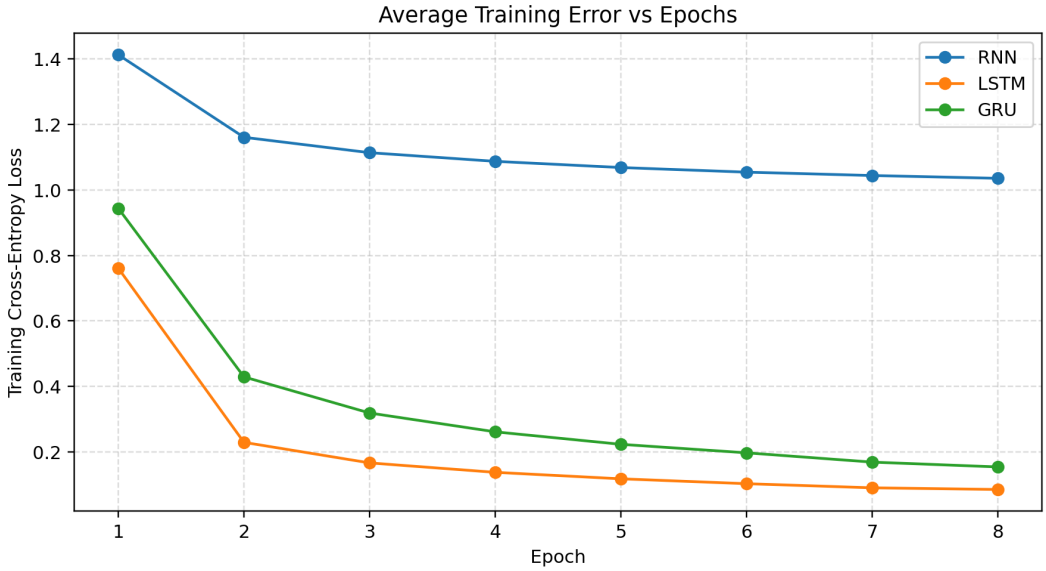


Figure 10. Average training cross-entropy loss versus epoch (8-epoch comparison).

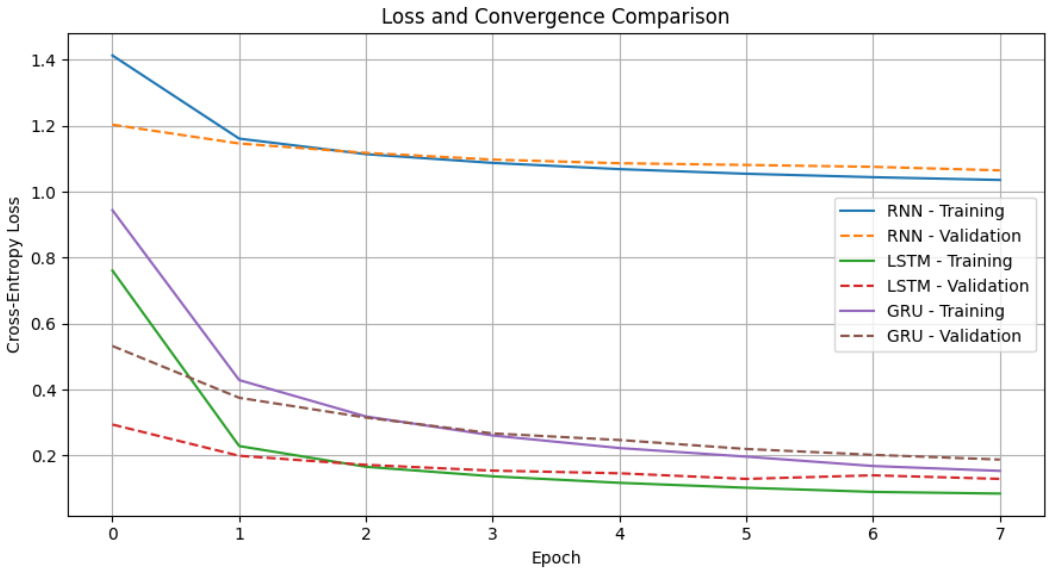


Figure 11. Training and validation loss curves (8-epoch comparison).

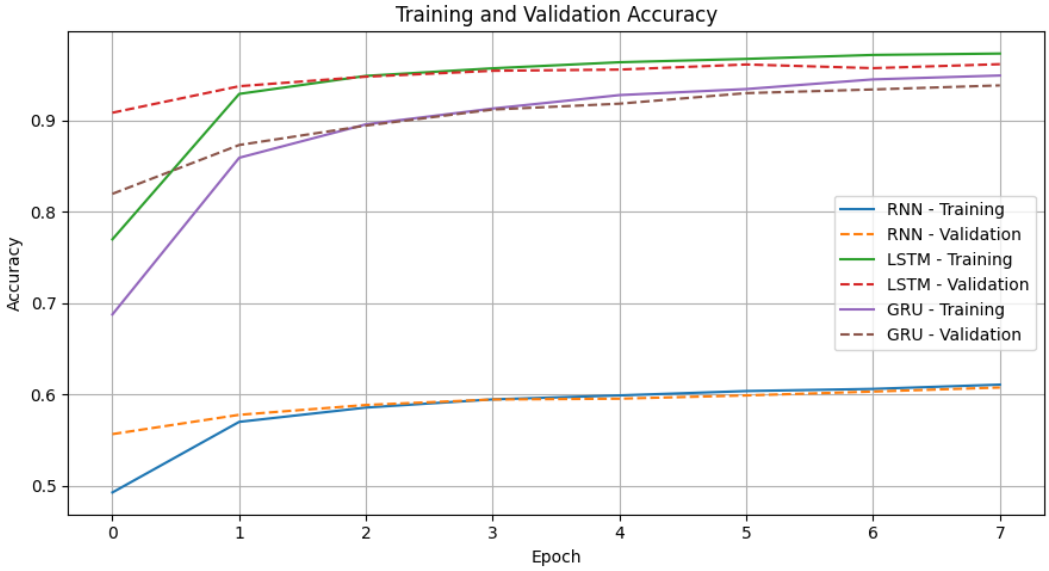


Figure 12. Training and validation accuracy curves (8-epoch comparison).

8. Hyperparameter Accuracy Summary (b)

Model	Units	LR	Batch	Train acc. %	Test acc. %
Vanilla RNN	64	0.001	128	60.71	59.93
Vanilla RNN	128	0.001	128	64.59	63.81
LSTM	64	0.001	128	96.81	95.52
LSTM	128	0.001	128	98.54	97.19
GRU	64	0.001	128	93.25	91.94
GRU	128	0.001	128	97.16	95.40

The highest recorded test accuracy in this separate five-epoch sweep is 97.19% for LSTM with 128 units. This is an exploratory sweep result: test accuracy was used to compare configurations, and the sweep's architecture (recurrent layer plus softmax output only) differs from the main model.

9. Main Model Comparison

Model	Train acc. %	Val acc. %	Train loss	Val loss	Test acc. %
Vanilla RNN	61.39	60.76	1.0194	1.0647	60.32
LSTM	97.72	96.14	0.0728	0.1292	95.90
GRU	95.56	93.82	0.1360	0.1878	93.73

Test accuracies are taken from the per-model classification reports (Cell 20). The validation-based selection is LSTM (96.14% validation accuracy), whose test accuracy is 95.90%. The later 27.34% result (Appendix A) is inconsistent because the notebook selects a best_model_name but does not reassign best_model to that model before evaluation.

10. Classification Reports and Confusion Matrices of All Three Models (c)

Each model below was evaluated on the same held-out test set of 9,000 images (900 per class). Rows of each confusion matrix are actual digits and columns are predicted digits.

10.1 Vanilla RNN (SimpleRNN) – test accuracy 60.32%

Class	Precision	Recall	F1-score	Support
0	0.4872	0.4244	0.4537	900
1	0.6896	0.7922	0.7373	900
2	0.8913	0.8656	0.8782	900
3	0.6073	0.6444	0.6253	900
4	0.4758	0.7089	0.5694	900
5	0.5943	0.5533	0.5731	900
6	0.6774	0.7467	0.7104	900
7	0.5144	0.4778	0.4954	900
8	0.4743	0.2667	0.3414	900
9	0.5917	0.5522	0.5713	900
accuracy			**0.6032**	**9000**
macro avg	**0.6003**	**0.6032**	**0.5955**	**9000**
weighted avg	**0.6003**	**0.6032**	**0.5955**	**9000**

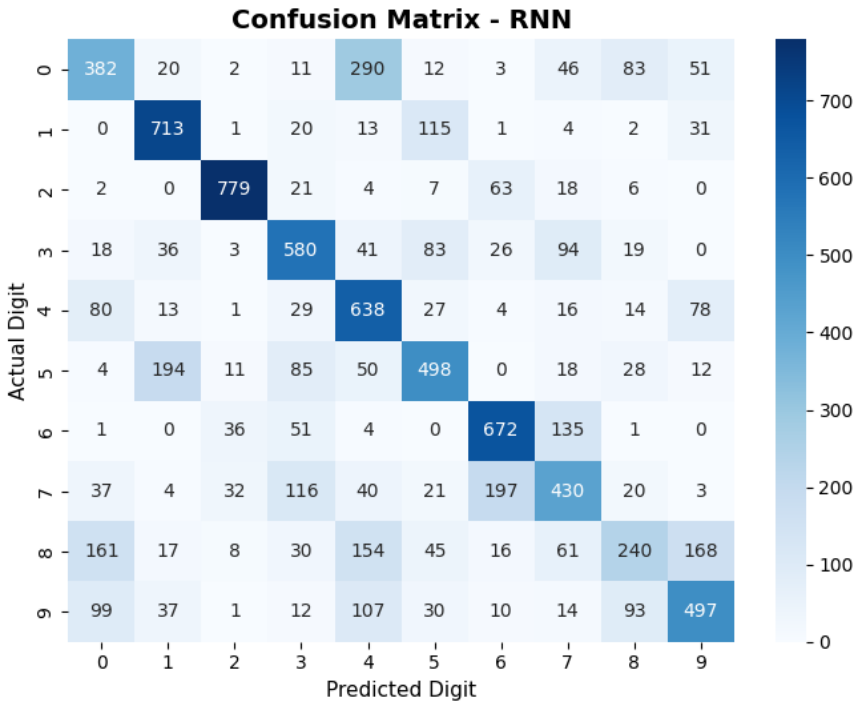


Figure 13. Confusion matrix – RNN (test set, 9,000 images).

10.2 LSTM – test accuracy 95.90%

Class	Precision	Recall	F1-score	Support
0	0.9677	0.9333	0.9502	900
1	0.9788	0.9767	0.9778	900
2	0.9857	0.9933	0.9895	900
3	0.9448	0.9500	0.9474	900
4	0.9469	0.9711	0.9589	900
5	0.9516	0.9611	0.9563	900
6	0.9382	0.9622	0.9501	900
7	0.9373	0.9300	0.9336	900
8	0.9728	0.9522	0.9624	900
9	0.9675	0.9600	0.9637	900
accuracy			**0.9590**	**9000**
macro avg	**0.9591**	**0.9590**	**0.9590**	**9000**
weighted avg	**0.9591**	**0.9590**	**0.9590**	**9000**

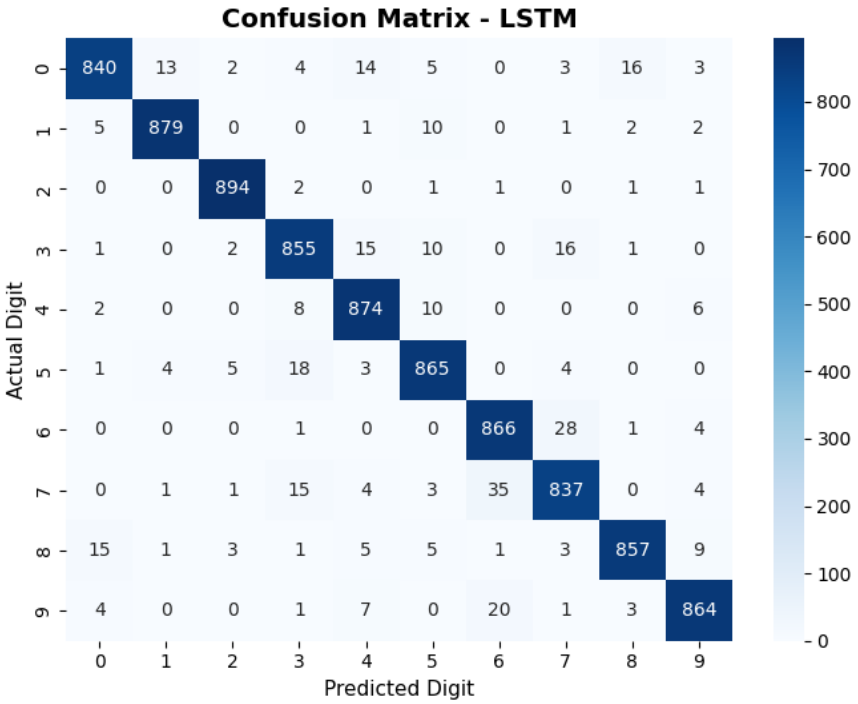


Figure 14. Confusion matrix – LSTM (test set, 9,000 images).

10.3 GRU – test accuracy 93.73%

Class	Precision	Recall	F1-score	Support
0	0.9412	0.9256	0.9333	900
1	0.9211	0.9600	0.9402	900
2	0.9846	0.9922	0.9884	900
3	0.9293	0.9056	0.9173	900
4	0.9210	0.9200	0.9205	900
5	0.9386	0.9167	0.9275	900
6	0.8937	0.9711	0.9308	900
7	0.9364	0.8667	0.9002	900
8	0.9606	0.9489	0.9547	900
9	0.9508	0.9667	0.9587	900
accuracy			**0.9373**	**9000**
macro avg	**0.9377**	**0.9373**	**0.9371**	**9000**
weighted avg	**0.9377**	**0.9373**	**0.9371**	**9000**

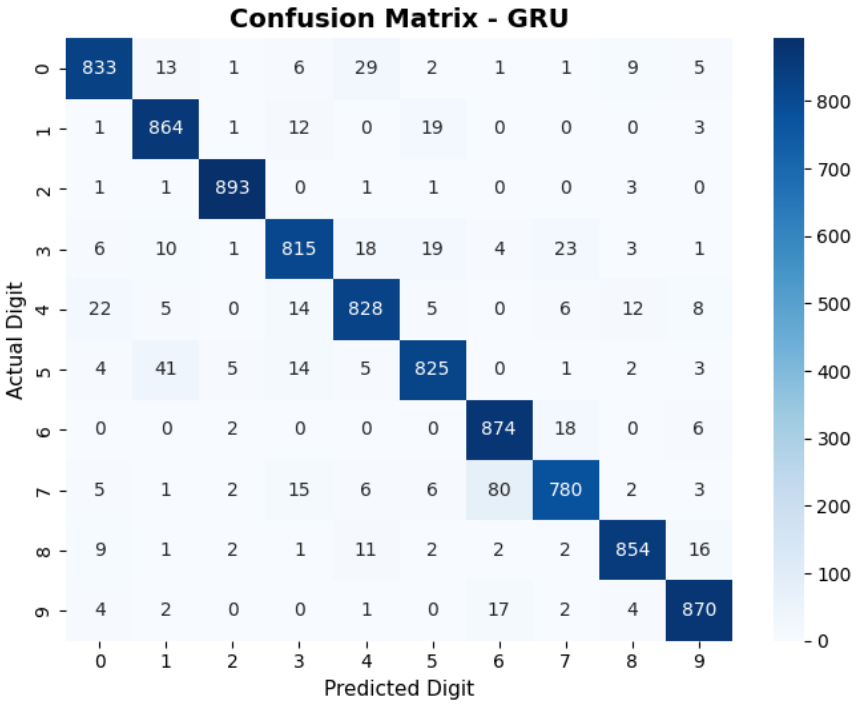


Figure 15. Confusion matrix – GRU (test set, 9,000 images).

10.4 Validation-selected LSTM: numeric confusion matrix

The main validation-selected LSTM has 95.90% test accuracy: 8,631 correct classifications from 9,000 test examples. Rows are actual digits; columns are predicted digits.

Actual \ Pred.	0	1	2	3	4	5	6	7	8	9
0	840	13	2	4	14	5	0	3	16	3
1	5	879	0	0	1	10	0	1	2	2
2	0	0	894	2	0	1	1	0	1	1
3	1	0	2	855	15	10	0	16	1	0
4	2	0	0	8	874	10	0	0	0	6
5	1	4	5	18	3	865	0	4	0	0
6	0	0	0	1	0	0	866	28	1	4
7	0	1	1	15	4	3	35	837	0	4
8	15	1	3	1	5	5	1	3	857	9
9	4	0	0	1	7	0	20	1	3	864

Largest notable off-diagonal counts for the LSTM: actual 6 predicted as 7 (28), actual 7 predicted as 6 (35), and actual 8 predicted as 0 (15). The 6/7 pair is the dominant confusion for LSTM and GRU alike, while Vanilla RNN spreads its errors over many pairs (e.g., 0 and 4, 8 and 9, 7 and 6) and has the weakest recall on digit 8.

11. Gradient Flow

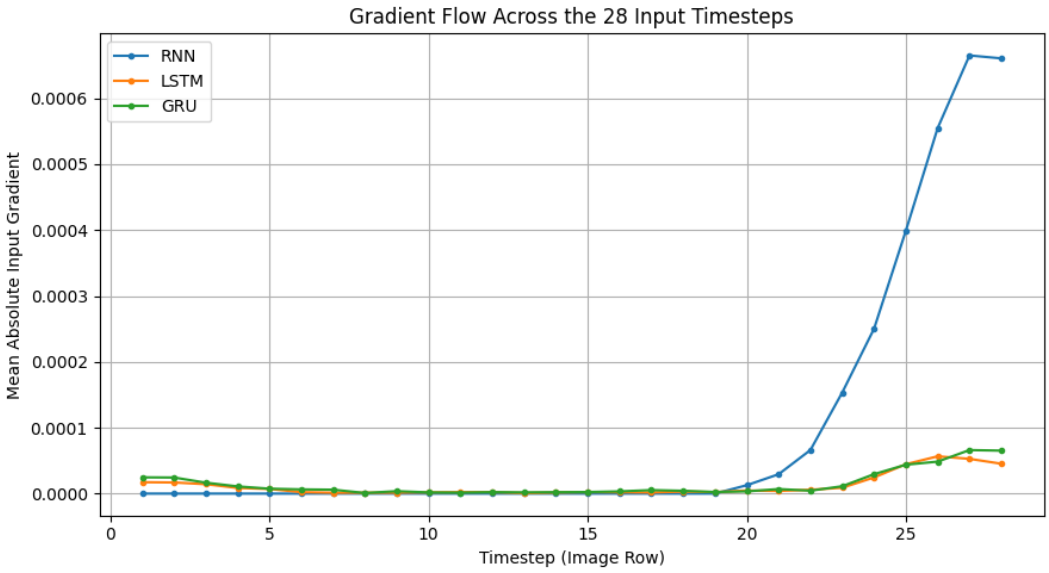


Figure 16. Mean absolute gradient with respect to input pixels by image-row timestep.

The diagnostic computes gradients of loss with respect to input pixels, not recurrent hidden states or recurrent weights. Recorded values are listed below. They do not prove that one model has superior recurrent gradient flow; they are affected by input scaling and model parameters. LSTM and GRU gates are designed to help preserve information, but this notebook does not directly measure that mechanism.

Model	Mean input gradient	Minimum input gradient
Vanilla RNN	1.00 × 10e-4	1.77e-23
LSTM	1.20e-5	7.06e-7
GRU	1.50e-5	9.25e-7

12. Comparative Observations and Empirical Analysis (e)

12.1 Performance

In the main 8-epoch comparison, LSTM is highest (96.14% validation; 95.90% test), followed by GRU (93.82% validation; 93.73% test), then Vanilla RNN (60.76% validation; 60.32% test). The separate sweep also shows LSTM and GRU outperforming Vanilla RNN. In the longer initial run, LSTM (98.01%) and GRU (97.57%) are close, while Vanilla RNN reaches only 74.08% validation accuracy.

12.2 Convergence

Training loss decreases across the eight epochs for all three models. LSTM falls from 0.7609 to 0.0846, GRU from 0.9440 to 0.1535, and Vanilla RNN from 1.4131 to 1.0355. LSTM learns fastest in this run, while the vanilla RNN remains at comparatively high loss. In the 40-epoch run the Vanilla RNN improves slowly and noisily (including a sharp validation dip near epoch 29) and had not converged when training ended, whereas LSTM and GRU triggered early stopping.

12.3 Gradient flow

Input-gradient magnitudes for the Vanilla RNN are concentrated almost entirely in the last few rows (timesteps), and are near zero for the early rows, which is consistent with the vanishing-gradient behaviour expected of a simple recurrent cell; LSTM and GRU show small but non-zero values across the early timesteps. Because the notebook measures input gradients rather than hidden-state gradients, this is suggestive, not conclusive.

12.4 Limitations

- Only one task/dataset (Kannada-MNIST) is included in the provided notebook; no results for a second task can be reported.
- The split-before-normalization issue must be fixed and the experiments rerun.
- The hyperparameter sweep did not save per-epoch histories for all six configurations.
- The final 27.34% result is caused by a stale/mismatched best_model variable; the per-model LSTM report gives 95.90%.
- Eight epochs provide a preliminary comparison, not proof of full convergence; the 40-epoch run reached no test evaluation in the notebook.

13. Conclusion

The executed notebook's validation-selected main model is LSTM, with 96.14% validation accuracy and 95.90% test accuracy. GRU is second, and Vanilla RNN performs substantially worse in this run. The separate five-epoch sweep records 97.19% test accuracy for a simpler 128-unit LSTM, and the longer initial run reaches 98.01% validation accuracy for LSTM. The report preserves these as distinct experiments. Correct normalization and rerunning the notebook are recommended before final submission.

Appendix A. Invalid Evaluation Output (stale best_model)

For transparency, the output of Cell 22 is shown. It reports "Selected model: LSTM" with test accuracy 27.34%, but best_model was never reassigned to the LSTM model, so the evaluated object is a different (stale) model. These numbers must not be used; the correct LSTM test results are in Section 10.2.

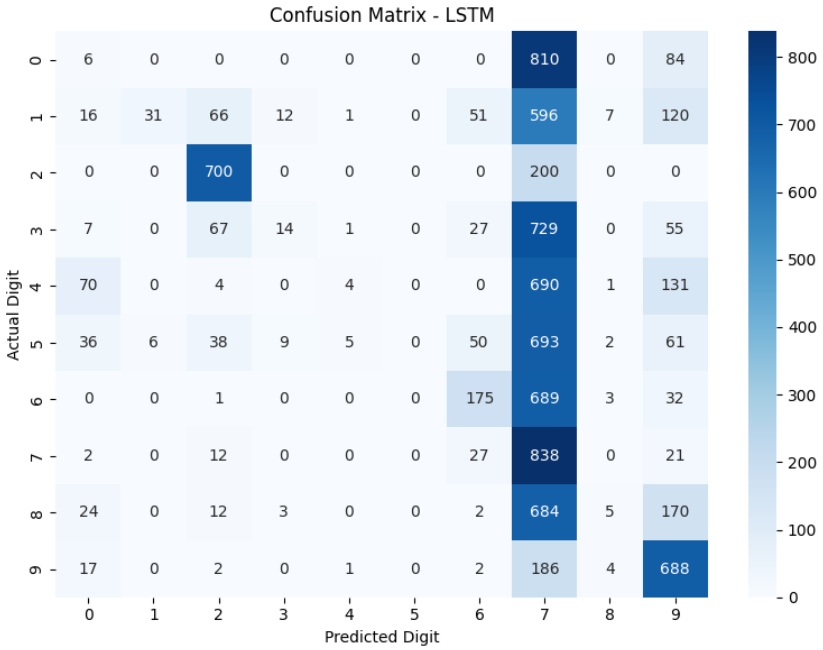


Figure A1. Confusion matrix from the stale best_model evaluation (27.34% test accuracy) – not valid for the LSTM.